

APIs e integrações

ATIVIDADE 2

Prof.^a Denilce

Disciplina: Programação Web

Alfred Domke Albach, 0030482511003

Sorocaba

Agosto/2026

1 INTRODUÇÃO	2
--------------------	---

2 DESENVOLVIMENTO.....	3
2.1 O QUE É UMA API.....	3
2.1.1 Conceito de API.....	3
2.1.2 Finalidade das APIs.....	3
2.1.3 Papel das APIs na comunicação entre sistemas.....	3
2.2 COMO FUNCIONAM AS APIs	3
2.2.1 Requisição e resposta.....	3
2.2.2 Cliente e servidor.....	3
2.2.3 Endpoints.....	3
2.2.4 Métodos HTTP	3
2.2.5 Formatos de dados: JSON e XML.....	3
2.3 TIPOS E ARQUITETURAS DE APIs.....	3
2.3.1 REST	3
2.3.2 SOAP	3
2.3.3 GraphQL	3
2.3.4 Comparação entre REST, SOAP e GraphQL.....	3
2.4 INTEGRAÇÃO ENTRE SISTEMAS.....	3
2.4.1 Conceito de integração de sistemas	3
2.4.2 Integração por meio de APIs	3
2.4.3 Comunicação entre diferentes aplicações e serviços	3
2.4.4 Benefícios das integrações.....	3
2.5 FERRAMENTAS PARA APIs.....	3
2.5.1 Postman	3
2.5.2 Swagger e OpenAPI.....	4
2.5.3 Ferramentas de desenvolvimento, testes e documentação	4
2.6 EXEMPLOS DE UTILIZAÇÃO DE APIs.....	4
2.6.1 Aplicações web	4
2.6.2 Aplicativos móveis.....	4
2.6.3 Sistemas de pagamento	4
2.6.4 Mapas e localização	4
2.6.5 Redes sociais	4
2.6.6 Sistemas corporativos	4

2.7 VANTAGENS E DESAFIOS DAS APIs.....	4
2.7.1 Reutilização e interoperabilidade.....	4
2.7.2 Escalabilidade.....	4
2.7.3 Segurança.....	4
2.7.4 Manutenção e versionamento	4
2.7.5 Dependência de serviços externos.....	4
3 CONCLUSÃO	4
4 REFERÊNCIAS	4

1 INTRODUÇÃO

No cenário tecnológico contemporâneo, as organizações deparam-se com ecossistemas de software altamente complexos e distribuídos, os quais necessitam cooperar de forma contínua para dar suporte aos processos de negócio. Nesse contexto, a **integração de sistemas (IS)** surge como um fenômeno no qual múltiplos sistemas de informação colaboram eletronicamente para atingir um objetivo específico. Essa prática é de interesse estratégico e econômico primordial, pois viabiliza a extensão da vida útil de sistemas legados e ajuda as organizações a responderem com agilidade às rápidas transformações de mercado. Frequentemente associada à **interoperabilidade** — definida como a habilidade de dois ou mais sistemas ou componentes trocarem informações e utilizarem essa informação de forma mútua —, a integração bem delineada estabelece canais de comunicação fluidos e reduz a criação de dependências tecnológicas rígidas entre as aplicações envolvidas.

Dentro desse cenário de conectividade, as **Interfaces de Programação de Aplicação (APIs — *Application Programming Interfaces*)** assumem um papel de relevância crítica. Elas atuam como interfaces de comunicação padronizadas que permitem que diferentes *softwares* e serviços se conectem, compartilhem dados e executem funcionalidades através de uma rede de computadores, simplificando significativamente o desenvolvimento de soluções digitais dinâmicas e interconectadas. O objetivo deste trabalho é apresentar uma análise acadêmica detalhada acerca das APIs e suas integrações. Para tanto, este documento abordará, de maneira estruturada, o conceito fundamental dessas interfaces, sua finalidade prática e o papel crucial que desempenham na intercomunicação de sistemas no ambiente tecnológico contemporâneo.

2 DESENVOLVIMENTO

Este capítulo estabelece a fundamentação teórica sobre o uso, a modelagem e a implementação de APIs na engenharia de *software* e na integração de sistemas de informação. Serão discutidos os aspectos conceituais que regem essas interfaces, os principais objetivos técnicos e operacionais de sua adoção no desenvolvimento moderno, bem como a mecânica estrutural por trás de sua capacidade de orquestrar a intercomunicação entre serviços e aplicações.

2.1 O QUE É UMA API

2.1.1 Conceito de API

A sigla **API** refere-se a ***Application Programming Interface*** (Interface de Programação de Aplicação). De modo geral, uma API consiste em um conjunto de regras ou protocolos que permitem que aplicativos de *software* se comuniquem entre si para trocar dados, recursos e funcionalidades. Sob uma perspectiva complementar, ela é definida como um contrato que especifica formalmente as condições e os meios pelos quais deve ocorrer a comunicação entre dois serviços distintos.

Em termos de engenharia de *software*, a API funciona como uma interface padronizada que permite a comunicação direta entre sistemas sem que um precise compreender como o outro foi internamente implementado. No contexto das modernas arquiteturas de sistemas distribuídos, uma API representa um componente de *software* autônomo e implantável de forma independente, que expõe uma funcionalidade útil para o sistema. A interação de qualquer cliente com o serviço ocorre por meio de pontos de acesso específicos chamados de ***endpoints***. Esses *endpoints* são portas de entrada que delimitam como as requisições devem ser codificadas e quais parâmetros específicos de cabeçalho (*header*) ou corpo (*body*) são requeridos. O cliente realiza uma requisição para o serviço através das condições impostas pela API, e o serviço envia uma resposta de volta nos padrões definidos no contrato.

2.1.2 Finalidade das APIs

A finalidade primordial de uma API é simplificar e padronizar o intercâmbio de dados e funcionalidades entre sistemas heterogêneos, eliminando a necessidade de retrabalho manual ou de compreensão detalhada do código-fonte e das regras internas do sistema provedor. Como o cliente não consegue visualizar o serviço que implementa a API, esta atua como uma **camada de abstração e segurança**, ocultando a complexidade lógica do *backend* e expondo apenas o contrato de utilização do serviço.

Adicionalmente, as APIs têm o propósito de promover o **desacoplamento de aplicações**, ou seja, a criação de serviços apartados e independentes. Essa separação garante que alterações ou evoluções tecnológicas em um microserviço ou módulo não impactem diretamente o funcionamento de outras partes da aplicação, reduzindo o risco de o sistema ficar indisponível. Sob a perspectiva da governança de TI, adotar um padrão bem definido para as APIs dos serviços é fundamental para evitar inconsistências em termos de formatos de entrada, saída, tratamento de erros e envelopes de resposta. Para assegurar essa consistência e automatizar a geração de códigos, documentações interativas e testes, as APIs podem ser modeladas por meio de linguagens de definição de interface (IDLs — *Interface Definition Languages*), como as especificações OpenAPI (via Swagger) e RAML.

2.1.3 Papel das APIs na comunicação entre sistemas

O papel das APIs na comunicação entre diferentes *softwares* e serviços manifesta-se tecnicamente como a base do mecanismo de **"Chamadas" (*call interface*)**, que constitui um dos três grandes grupos de tecnologias de integração de sistemas, juntamente com as "Mensagens" (*messaging*) e a "Transferência de Arquivos/Compartilhamento de Dados". Nesse papel, as aplicações provedoras disponibilizam uma interface chamável por meio da qual um cliente envia requisições estruturadas e recebe respostas correspondentes.

As APIs exercem uma função crítica no suporte à interoperabilidade de dados e funcionalidades, permitindo a comunicação fluida sem que se crie uma dependência tecnológica rígida entre os sistemas conectados. Elas suportam tanto comunicações **síncronas**, nas quais o cliente envia uma requisição e deve aguardar a resposta do servidor em tempo real (como ocorre predominantemente em arquiteturas REST e GraphQL), quanto comunicações **assíncronas**, nas quais a transmissão de dados, comandos, consultas e eventos é efetuada por meio de filas e mensageria (*message brokers*).

As fontes bibliográficas trazem diversos exemplos práticos que ilustram esse papel de comunicação:

- **Integração no Sistema Único de Saúde (SUS):** Na federação de dados clínicos desenvolvida entre Recife e Ebserh, barramentos de serviços com gestão de APIs foram implementados sob padrões internacionais de interoperabilidade (como HL7 FHIRv4 e OpenEHR). Por meio de conectores e APIs específicas, dados clínicos de prontuários eletrônicos (e-SUS APS e AGHU), regulação assistencial (via API do Sisreg), imunizações (via API do Sipni) e resultados de exames de diagnóstico (LISNet) são orquestrados e compartilhados em tempo real, superando a fragmentação histórica das informações de saúde dos cidadãos.
- **Integração de Sistemas Corporativos (Caso GVT):** No suporte à operação da empresa de telecomunicações GVT, as APIs e as interfaces de chamada de procedimento (*call interface*) atuaram em conjunto com sistemas de mensageria para integrar SIs críticos de CRM, faturamento (*billing*), provisionamento de rede e geolocalização. A integração integrada permitiu automatizar o fluxo do processo de instalação de novas linhas de minutos para dias, reduzindo os esforços manuais de provisionamento e as inconsistências em bases de dados.
- **Comunicação Baseada em API Web em Aplicações Reais:** O desenvolvimento do sistema "Apoio à Revisão Sistemática" exemplifica a comunicação clássica entre uma interface de usuário (UI) responsiva em Angular e uma API desenvolvida em C# com o framework .NET Core. Essa comunicação utiliza o padrão arquitetural REST (via protocolo HTTPS) para a transferência de dados representacionais no formato leve JSON, assegurando que o processamento pesado de remoção de artigos duplicados seja executado no servidor enquanto mantém a memória do servidor otimizada para múltiplos usuários.
- **Modelagem de Dados e Latência de Comunicação (PokéAPI):** Em estudos comparativos de desempenho acadêmico, APIs estruturadas sob os padrões REST e GraphQL foram criadas e testadas com base no banco de dados adaptado do PokéAPI, envolvendo tabelas de entidades como *Pokemon*, *Ability*, *Item* e *PokemonForm*. O teste analisa como a comunicação pode ser configurada para requisições de entidade única ou agregadas em múltiplos endpoints, impactando de forma mensurável a latência e a eficiência do tráfego de rede observado pelo cliente final.

2.2 COMO FUNCIONAM AS APIs

As Interfaces de Programação de Aplicação (APIs) funcionam como contratos formais de comunicação que padronizam e simplificam o intercâmbio de dados e funcionalidades entre sistemas distintos, sem a necessidade de um *software*

compreender os detalhes da implementação interna do outro. A operação básica e estrutural por trás de uma API baseia-se no fluxo contínuo de tráfego de rede estabelecido entre dois componentes lógicos principais, regido por mensagens padronizadas e regras que determinam como os serviços expostos devem ser consumidos e respondidos.

2.2.1 Requisição e resposta

Conceito: O fluxo de comunicação de uma API baseia-se no binômio **requisição (request)** e **resposta (response)**. A requisição consiste na mensagem estruturada que o cliente envia ao servidor sob as condições estabelecidas na especificação da API. Por sua vez, a resposta constitui a mensagem de retorno gerada pelo servidor, a qual devolve ao cliente os dados ou status resultantes do processamento.

Função: Coordenar e disciplinar a troca transacional de informações em tempo real ou de forma programada. Em arquiteturas REST, o servidor complementa a resposta enviando códigos de retorno HTTP (como o código 200 para sucesso comum, 201 para a criação bem-sucedida de um recurso, 401 para falha de autenticação ou 500 para erro no servidor).

Exemplo prático: Quando um aplicativo cliente necessita recuperar uma lista de livros de um autor com identificador 32, ele dispara uma requisição sob o método GET para a URI <http://www.biblioteca.com/api/v1/autores/32/livros>. O servidor processa a solicitação e envia de volta uma resposta com o status de sucesso 200 acompanhado do arquivo contendo os dados dos livros do autor.

2.2.2 Cliente e servidor

Conceito: Reflete a separação clara de responsabilidades entre os componentes que se interconectam através da rede. O **cliente** é definido como a aplicação ou componente que inicia a comunicação por meio do envio de uma requisição. O **servidor** é o software encarregado de escutar, validar, processar e responder à referida requisição.

Função: Promover o desacoplamento e a escalabilidade. Ao isolar a interface do usuário (lado do cliente) da lógica de negócios e persistência de dados (lado do servidor), o cliente ganha portabilidade para rodar em variadas plataformas (como computadores,

tablets ou smartphones) sem exigir alterações no servidor. O servidor, por sua vez, pode ser escalado de forma independente.

Exemplo prático: No sistema "Apoio à Revisão Sistemática", a Interface de Usuário (UI) atua como o **cliente**, sendo desenvolvida em TypeScript (via framework Angular). Esta UI cliente envia requisições HTTPS para a API no **servidor** (desenvolvida em C# com .NET Core), que executa os processamentos pesados (como a exclusão automática de duplicatas de artigos) e interage com o banco de dados Microsoft SQL Server. Outro exemplo prático reside no Aplicativo de Gestão para Hospitais Universitários (AGHU) da Ebserh, que consome dados clínicos de outros hospitais e de barramentos de interoperabilidade.

2.2.3 Endpoints

Conceito: Os **endpoints** (pontos de acesso) são os locais lógicos específicos disponibilizados pela API de um serviço para permitir a comunicação com o mundo externo. Na arquitetura REST baseada em HTTP, esses endpoints são mapeados em URLs (*Uniform Resource Identifiers*) que servem como identificadores únicos para os recursos do sistema.

Função: Delimitar as regras de acesso ao recurso por ele representado, indicando como as requisições devem ser codificadas e quais parâmetros específicos de cabeçalho (*header*), corpo (*body*), caminho (*path*) ou consulta (*query*) são obrigatoriamente requeridos para que uma transação ocorra de maneira válida.

Exemplo prático: Na especificação de uma API em C#, o endpoint `/países` expõe as funcionalidades de obter a lista de países cadastrados, e o endpoint `/países/{nome}` utiliza uma variável de caminho (*path*) para buscar um país específico pelo seu nome. Um exemplo de modelagem diferente ocorre na API-R1 (usada em testes com dados da PokéAPI), que mapeia recursos no formato de "um endpoint por entidade" (exigindo que o cliente chame endpoints diferentes para Pokémons e depois suas habilidades).

2.2.4 Métodos HTTP

Conceito: São elementos semânticos que indicam de forma explícita ao servidor o tipo de processamento lógico que se deseja executar sobre o recurso identificado.

Função: Fornecer uma anotação padronizada que dão suporte direto à implementação de operações do tipo CRUD (Criação, Leitura, Atualização e Deleção) sobre os recursos expostos. As regras básicas são:

- **GET:** Utilizado para obter ou ler a representação de recursos no sistema. Conceitualmente, requisições sob o método GET jamais devem conter dados em seu corpo (*body*).
- **POST:** Utilizado para solicitar a criação de novos recursos.
- **PUT:** Utilizado para atualizar completamente um recurso, exigindo o envio de todas as propriedades do objeto no corpo da requisição.
- **PATCH:** Utilizado para atualizações parciais de atributos específicos.
- **DELETE:** Utilizado para remover recursos específicos do sistema.

Exemplo prático: Em uma API de catálogo, o cliente executa uma consulta de busca enviando o método GET para o endpoint `/produtos`. Caso queira incluir um novo item no estoque, ele enviará os parâmetros no corpo da requisição sob o método POST para o mesmo endpoint `/produtos`. Se houver a necessidade de remover um produto, o método DELETE será enviado apontando para o caminho do produto específico no endpoint, como `/produtos/9c88b109-ad01-4606`.

2.2.5 Formatos de dados: JSON e XML

Conceito: São formatos estruturados de mídias não binárias comumente utilizados no corpo das requisições e respostas para representar e transportar informações entre as aplicações.

Função: Prover um padrão de serialização textual que seja totalmente independente de linguagem de programação, viabilizando a interoperabilidade entre sistemas heterogêneos (como servidores em Java ou .NET e aplicações cliente em JavaScript ou Python).

- **JSON (*JavaScript Object Notation*):** É uma linguagem de formatação textual leve estruturada com base em par chave-valor e listas. É o formato de intercâmbio de dados mais popular no desenvolvimento moderno de APIs baseadas no padrão REST e GraphQL, pois requer menor processamento para codificação e decodificação pelas aplicações, otimizando o consumo de rede e a latência.
- **XML (*eXtensible Markup Language*):** É um formato de dados hierárquico estruturado com base em tags customizadas e definições de namespaces para eliminar ambiguidades. É a mídia padrão adotada no protocolo

SOAP, operando estruturalmente em camadas como envelope, header e body.

Exemplo prático: O sistema de apoio metodológico sysrev.azurewebsites.net trafega todas as informações clínicas e cadastrais de seus revisores em formato JSON por meio de requisições HTTP REST, mantendo o consumo de memória do servidor minimizado e garantindo renderização rápida em tela. Por outro lado, requisições SOAP para APIs de simulação de cálculos matemáticos (como <http://www.dneonline.com/calculator.asmx?WSDL>) trafegam seus payloads utilizando XML, estruturando a operação de soma e suas variáveis de entrada dentro da tag <tem:Add> envolvida pelas camadas de <soapenv:Body> e <soapenv:Envelope>.

2.3 TIPOS E ARQUITETURAS DE APIs

Com o aumento da conectividade e da complexidade das soluções digitais contemporâneas, diversas tecnologias e paradigmas foram aplicados no desenvolvimento de APIs Web, exigindo que cada alternativa seja cuidadosamente analisada à luz dos recursos oferecidos e das necessidades específicas de seus consumidores. A escolha entre os diferentes modelos de construção de APIs aplicados a arquiteturas modernas traz para o desenvolvedor a necessidade de conhecer profundamente esses padrões para saber onde e como utilizá-los de forma adequada. Cada implementação apresenta características e limitações distintas que se revelam mais vantajosas em determinados cenários do que em outros. Entre as principais arquiteturas e padrões síncronos de APIs abordados pela literatura de engenharia de software destacam-se o REST, o SOAP e o GraphQL, com o REST posicionando-se como a abordagem mais popular no mercado atual.

2.3.1 REST

O padrão arquitetural REST (*Representational State Transfer*) foi proposto originalmente no ano 2000 na tese de doutorado de Roy Thomas Fielding. A fundamentação do REST consiste na exploração de arquiteturas de aplicações baseadas em rede sob a ótica de restrições arquiteturais obrigatórias aplicadas de maneira incremental sobre os elementos do sistema. De acordo com Fielding, a arquitetura é definida por seis restrições principais:

1. **Arquitetura Cliente-Servidor:** Estabelece uma clara separação de responsabilidades entre os componentes que se comunicam através da rede. O cliente atua como o software que inicia a comunicação disparando requisições, enquanto o servidor recebe, processa e atende a essas solicitações, viabilizando que múltiplos clientes heterogêneos consumam recursos de um único servidor. Essa separação eleva de forma significativa a portabilidade da interface do usuário em diferentes plataformas e promove a escalabilidade ao reduzir a complexidade interna do servidor.
2. **Comunicação Sem Estado (Stateless):** Determina que as interações ocorram sem que o servidor precisar armazenar ou consultar informações sobre o contexto da sessão do cliente. Cada requisição enviada ao servidor deve conter, obrigatoriamente, todas as informações necessárias para que a operação seja processada e concluída. Essa restrição aumenta a visibilidade do sistema, simplifica a recuperação de falhas parciais (confiabilidade) e otimiza a escalabilidade, facilitando a alocação e liberação de recursos do servidor.
3. **Uso de Cache:** Exige que as mensagens do servidor informem explicitamente ao cliente se o dado retornado pode ser armazenado localmente em cache. A utilização dessa técnica evita a execução de novas requisições desnecessárias para o servidor, embora possa reduzir a confiabilidade dos dados caso a resposta em cache apresente discrepâncias em relação ao estado atualizado do servidor.
4. **Interface Uniforme:** Simplifica a arquitetura geral do sistema e melhora a visibilidade das interações entre os componentes, permitindo que as implementações dos serviços evoluam de maneira independente devido ao desacoplamento. No entanto, a uniformidade impõe uma perda de desempenho, pois as informações são transferidas de maneira padronizada e geral, em vez de formatos específicos otimizados para uma aplicação específica. Compõe-se de quatro elementos fundamentais: identificação de recursos, manipulação de recursos, mensagens autodescritivas que orientam o cliente sobre como lidar com o retorno, e a disponibilização de hyperlinks de hipermídia (HATEOAS).
5. **Sistema em Camadas:** Organiza a infraestrutura de serviços em camadas lógicas distribuídas, onde cada serviço possui conhecimento restrito apenas às suas camadas adjacentes. Essa separação reduz a complexidade global do sistema, permite o balanceamento de carga de rede através de servidores intermediários, isola componentes legados e propicia a evolução independente dos serviços. Como desvantagem técnica, o modelo em camadas introduz overhead de processamento e eleva a latência final da comunicação.
6. **Código Sob Demanda (Opcional):** Permite que o cliente faça o download de código executável do servidor (como classes Java ou applets), liberando o cliente de possuir determinados recursos pré-instalados de fábrica e estendendo dinamicamente as capacidades do sistema, embora reduza a visibilidade geral do fluxo de execução.

Do ponto de vista prático e de implementação, nas APIs baseadas em REST, cada recurso de sistema (documento, imagem ou entidade) é representado e identificado de maneira única por meio de uma URI (*Uniform Resource Identifier*). As operações a serem realizadas sobre esses recursos são indicadas pelos verbos HTTP, os quais conferem anotação semântica explícita para cada funcionalidade da API: GET (leitura, que conceitualmente jamais deve apresentar corpo de requisição), POST (criação), PUT (atualização completa do objeto), PATCH (atualização parcial) e DELETE (remoção). A indicação do sucesso ou falha da transação é representada pelos códigos de retorno HTTP, divididos em cinco categorias: informativos (1xx), sucesso (2xx), redirecionamentos (3xx), erros no cliente (4xx) e erros no servidor (5xx). Os formatos de representação de dados mais comuns trafegados no corpo (*body*) das requisições e respostas REST são o JSON e o XML. Como a arquitetura REST não possui uma linguagem de definição de interface (IDL) nativa, a indústria de software utiliza especificações externas como a OpenAPI Specification (OAS), amplamente documentada por ferramentas como o Swagger, ou o RAML para definir seus contratos operacionais.

2.3.2 SOAP

O padrão SOAP (*Simple Object Access Protocol*) constitui uma especificação de modelagem de APIs síncronas que utiliza estritamente mídias não binárias e o protocolo HTTP/1 para a transmissão de dados. Diferente de estilos arquiteturais flexíveis, o SOAP é um protocolo formal caracterizado pelo tráfego de dados exclusivamente estruturados em formato XML (*eXtensible Markup Language*). Por possuir uma arquitetura considerada pesada e rígida, seu padrão não é mais amplamente recomendado para o desenvolvimento de novas aplicações na internet, sendo classificado como uma tecnologia defasada para o cenário moderno de desenvolvimento. No entanto, por sua robustez contratual histórica, as APIs SOAP são frequentemente encontradas em sistemas legados corporativos robustos, como os de instituições bancárias e financeiras.

Estruturalmente, as mensagens trafegadas no padrão SOAP são compostas por três camadas fundamentais organizadas hierarquicamente:

1. **Envelope:** Camada obrigatória que engloba toda a mensagem em um único arquivo XML. Ela possui e especifica o conteúdo geral da mensagem, delimitando o início e o fim do documento, além de conter os detalhes técnicos necessários para o processamento da mensagem.
2. **Regras de Codificação (Rules):** Contém as diretrizes formais que definem como os tipos de dados customizados do sistema devem ser codificados.

3. **Aplicação:** Especifica o comportamento prático do envelope e as regras de mapeamento de dados para as chamadas e respostas geradas pelas aplicações cliente e servidor.

O envelope SOAP pode conter um elemento opcional chamado **Header** (cabeçalho), que deve ser o primeiro subelemento do envelope caso esteja presente. O Header é utilizado para codificar dados que vão além da regra de negócio em si, como de autenticação, controle de segurança e diretrizes de processamento intermediário (como o atributo action, que permite enviar dados consumidos por intermediários antes que a mensagem atinja o destinatário final). Independentemente de haver um Header, a mensagem SOAP deve conter obrigatoriamente um subelemento chamado **Body** (corpo), que abriga o payload principal de dados da transação. Para a serialização de dados, o protocolo utiliza o atributo encodingStyle associado ao envelope ou ao corpo, definindo as regras de codificação por meio de um *namespace* (URI que elimina ambiguidades de interpretação das tags XML). O contrato e o comportamento esperado da API SOAP, detalhando funções, parâmetros e respostas, são formalmente descritos em arquivos do tipo WSDL (*Web Services Description Language*).

As principais diferenças do SOAP em relação ao REST referem-se à sua natureza estrutural e semântica:

- **Protocolo versus Estilo Arquitetural:** O SOAP é um protocolo rígido com regras de mensagens estritas, enquanto o REST é um estilo arquitetural flexível governado por um conjunto de restrições.
- **Semântica de Acesso:** O REST utiliza múltiplos verbos HTTP semânticos (GET, POST, PUT, DELETE) mapeados diretamente em URIs específicas para cada recurso. O SOAP, por sua vez, unifica a comunicação em um único endpoint, mapeando a ação a ser executada e suas variáveis dentro de estruturas XML específicas no corpo da requisição (por exemplo, dentro de tags como <tem:Add>).
- **Formato de Tráfego:** O REST prioriza formatos leves e eficientes de serialização como o JSON (embora suporte XML). O SOAP é estritamente limitado ao XML envolvido por envelopes pesados, o que aumenta o tráfego de rede e o custo computacional de codificação e decodificação.

2.3.3 GraphQL

O GraphQL constitui um motor de execuções (*execution engine*) e uma linguagem de consulta (*query language*) para APIs Web de tipagem forte que funciona de maneira

independente de linguagem de programação ou base de dados. Criado pelo Facebook em 2012 e lançado como open-source em 2015, ele é mantido atualmente sob a governança da GraphQL Foundation. Diferente das abordagens tradicionais que enxergam dados em estruturas de tabelas ou recursos isolados, o GraphQL parte do princípio de design de representação de dados baseado em uma estrutura de grafos direcionados, modelando as entidades como vértices e suas conexões como arestas.

A comunicação em GraphQL baseia-se na centralização de todas as interações da aplicação em uma **única URI** através dos métodos HTTP GET ou POST, abandonando a dependência de múltiplos endpoints. Nas chamadas que utilizam o POST, um documento JSON contendo a consulta e suas variáveis é enviado no corpo da requisição. O servidor GraphQL recebe a consulta escrita na linguagem própria do GraphQL, realiza validações de sintaxe de acordo com as regras estabelecidas no contrato, e encaminha a requisição para funções internas chamadas **resolvers**, que são encarregadas de recuperar os dados solicitados dos objetos no banco de dados e retornar um JSON formatado no corpo da resposta.

A arquitetura GraphQL é baseada em um contrato formal chamado **schema**, que define as operações e tipos de dados com tipagem forte. A linguagem suporta três operações principais:

1. **Query:** Operação síncrona de consulta de dados, cuja semântica se assemelha a uma instrução SQL SELECT.
2. **Mutations:** Operações destinadas à gravação, alteração ou exclusão de dados, análogas às instruções SQL INSERT, UPDATE e DELETE.
3. **Subscriptions:** Mecanismo utilizado para monitorar alterações de dados em tempo real, notificando o cliente de maneira contínua quando determinadas operações ocorrem.

As principais diferenças entre o GraphQL e a arquitetura REST abrangem a flexibilidade e a eficiência do consumo de dados:

- **Múltiplos Endpoints versus URI Única:** O REST trata cada entidade como um recurso isolado mapeado em uma URI distinta (como /pokemons ou /habilidades). O GraphQL consolida toda a API em uma única URI, expondo todas as suas capacidades por meio de um schema autodescritivo.
- **Resolução de Over-fetching e Under-fetching:** Em APIs REST, as respostas dos endpoints possuem estruturas fixas e predefinidas pelo servidor, resultando frequentemente em *over-fetching* (envio de dados adicionais que o cliente não utilizará) ou *under-fetching* (retorno de dados insuficientes, obrigando o cliente a realizar múltiplas chamadas consecutivas para completar uma visualização). O GraphQL resolve esses problemas ao transferir para o cliente a capacidade de especificar exatamente quais campos e conexões devem ser retornados na resposta.

- **Tratamento de Erros:** O REST invalida completamente a transação quando ocorre uma falha parcial, retornando um status de erro HTTP geral (como 500 ou 4xx). O GraphQL facilita o tratamento de erros em tempo de execução ao permitir falhas parciais; na ocorrência de problemas, ele devolve um objeto contendo os dados recuperados com sucesso ao lado de uma lista detalhada de erros parciais observados.
- **Controle de Versão:** O REST requer versionamento explícito (frequentemente numérico na URI, como /v1/), dificultando para os clientes rastrear as evoluções. O GraphQL evolui de maneira contínua e sem versionamento através de registros de schema comparáveis ao controle de versões do GIT.
- **Desempenho e Latência de Rede:** Pesquisas empíricas comparativas demonstram que o número de entidades envolvidas não determina de maneira absoluta qual tecnologia será mais veloz, dependendo diretamente da quantidade de requisições de rede executadas. Quando os dados podem ser obtidos em uma única requisição (situação ideal do REST com endpoint consolidado), o REST apresenta desempenho **11% a 14% superior** ao GraphQL, devido ao overhead computacional de processamento gerado pelo motor de execução do GraphQL. No entanto, em cenários comuns onde o REST exige múltiplas chamadas consecutivas para recuperar entidades relacionadas (problema N+1 das APIs públicas com endpoints por entidade), o GraphQL reduz os tempos de rede e apresenta desempenho **2701% melhor** do que o REST.

2.3.4 Comparação entre REST, SOAP e GraphQL

A tabela a seguir apresenta uma síntese comparativa objetiva entre REST, SOAP e GraphQL, estruturada a partir de suas características técnicas centrais de implementação, vantagens operacionais, limitações críticas e situações ideais de adoção prática:

- **Padrão REST:**
 - *Características:* Estilo arquitetural orientado a recursos (URIs); comunicações obrigatoriamente sem estado (*stateless*); uso de múltiplos verbos semânticos HTTP; múltiplos endpoints; e suporte a formatos flexíveis como JSON ou XML.
 - *Vantagens:* Arquitetura extremamente popular com ecossistema rico de ferramentas estáveis e vasta comunidade ativa; facilidade de aprendizado e manutenção de código; suporte nativo e eficiente a cache de rede; e

maior velocidade (11% a 14% mais rápido que o GraphQL) em cenários de requisição única.

- *Limitações:* Respostas fixas e inflexíveis definidas exclusivamente pelo servidor; alta incidência de problemas de *over-fetching* e *under-fetching*; e vulnerabilidade ao gargalo do problema de consulta N+1 em rede (gerando latências severas).
- *Situação de Uso:* Aplicações tradicionais do tipo CRUD; microsserviços internos que demandam comunicações diretas síncronas simples; e sistemas de informação onde o cache de rede e do servidor desempenha um papel crítico para a estabilidade.

- **Padrão SOAP:**

- *Características:* Protocolo de comunicação formal e rígido; tráfego de dados restrito a mensagens XML; estrutura de encapsulamento pesada com Envelope, Header e Body; definição de interface via arquivo WSDL.
- *Vantagens:* Contrato de interface formal, rígido e padronizado; alto nível de segurança operacional e regras estritas de transação corporativa.
- *Limitações:* Tecnologia obsoleta com alto consumo de rede devido a payloads pesados; complexidade acentuada de codificação e decodificação dos envelopes XML no cliente e servidor.
- *Situação de Uso:* Integrações de sistemas legados que exigem alta conformidade transacional; e sistemas corporativos de missão crítica (como as plataformas de comutação financeira ou bancárias).

- **Padrão GraphQL:**

- *Características:* Linguagem de consulta e motor de execução de dados baseado em grafos (vértices e arestas); ponto de acesso centralizado em uma única URI (endpoint único); schema autodescritivo fortemente tipado; e uso de resolvers no servidor.
- *Vantagens:* Altíssima flexibilidade nas consultas; eliminação completa de *over-fetching* e *under-fetching* (o cliente escolhe os campos); recuperação de dados de múltiplas entidades relacionadas em uma única chamada (2701% de superioridade em relação a múltiplas chamadas REST); tratamento nativo de erros parciais; e schema autodescritivo livre de controle manual de versão.
- *Limitações:* Processamento computacional pesado de validação e interpretação de consultas no servidor; e maior complexidade técnica e tempo de implementação para a equipe de desenvolvimento.
- *Situação de Uso:* APIs públicas e abertas onde é impossível prever todas as formas de consulta dos usuários; aplicações com interfaces de usuário complexas que demandam dados hierárquicos e aninhados; e sistemas que operam em canais com limitações severas de rede (dispositivos móveis), onde é crítico evitar múltiplas requisições sequenciais.

2.4 INTEGRAÇÃO ENTRE SISTEMAS

O desenvolvimento e a evolução dos sistemas de informação nas organizações modernas geram uma necessidade contínua de conectividade e troca de dados. Esta seção descreve as bases teóricas e práticas da integração de sistemas, destacando o papel das APIs nesse processo, os modelos de comunicação síncronos e assíncronos estabelecidos e os benefícios organizacionais e técnicos observados.

2.4.1 Conceito de integração de sistemas

A integração de sistemas (IS) caracteriza-se como um fenômeno tecnológico e operacional no qual mais de um sistema de informação colabora eletronicamente para atingir um determinado objetivo. De acordo com a literatura de engenharia de software, essa colaboração pode ocorrer tanto entre sistemas de informação completamente diferentes quanto entre módulos ou microsserviços integrados de um único sistema distribuído, quando há a necessidade de que estes interajam para atender a requisitos e regras específicas de um negócio. A prática da integração de sistemas é de interesse estratégico e econômico primordial para as organizações, uma vez que viabiliza a extensão da vida útil de sistemas legados (SL) que realizam contribuições críticas para o sucesso operacional, mas que, por outro lado, oferecem forte resistência à mudança e prejudicam a agilidade organizacional perante as transformações do ambiente externo.

É de fundamental importância para os profissionais de tecnologia discernir o conceito de integração do conceito de interoperabilidade, uma vez que na literatura científica eles são frequentemente tratados de forma distinta. **Enquanto a integração propriamente dita refere-se ao processo físico de conectar dois ou mais sistemas de forma a gerar uma dependência tecnológica ou acoplamento rígido entre eles, a interoperabilidade consiste na habilidade de dois ou mais sistemas ou componentes trocarem informação e serem capazes de utilizar essa informação trocada de forma mútua e útil, sem a geração de uma dependência tecnológica rígida.** Portanto, a busca por soluções de integração modernas visa, preferencialmente, estabelecer a interoperabilidade das bases de dados e aplicações para assegurar um fluxo consistente de dados.

2.4.2 Integração por meio de APIs

As tecnologias de integração de sistemas de informação baseiam-se em recursos computacionais que podem ser classificados a partir de três mecanismos fundamentais de tráfego e acesso: as mensagens (*messaging*), nas quais as aplicações trocam dados por meio de filas de mensagens; a transferência de arquivos ou compartilhamento de dados (*data access/file transfer*), em que as aplicações são integradas via acesso direto às mesmas bases de dados ou efetuam cargas em lote (*batch*); e as chamadas (*call interface*), nas quais as aplicações disponibilizam uma interface padrão que pode ser chamada programaticamente. **As Interfaces de Programação de Aplicação (APIs) enquadram-se na categoria de chamadas (*call interface*), fornecendo uma interface de comunicação padronizada que permite que diferentes softwares e serviços se conectem e compartilhem dados e funcionalidades de forma simplificada.**

Sob a ótica de projeto e arquitetura de software, a API atua como um contrato formal que especifica como deve ocorrer a comunicação entre dois serviços distintos, sem que a aplicação cliente precise visualizar ou compreender os detalhes da implementação interna ou do código-fonte do serviço provedor. O uso de APIs atua como um elemento central para promover o desacoplamento das aplicações, permitindo a separação em serviços e microsserviços independentes e autônomos. **Esse modelo evita o surgimento da "integração do tipo espaguete" (*spaghetti integration*), típica da abordagem tradicional de integração ponto a ponto em que analistas criavam conexões locais rígidas e específicas diretamente entre sistemas legados e novas aplicações, resultando na perpetuação do legado e no aumento severo dos custos e riscos de substituição ou manutenção desses sistemas.**

2.4.3 Comunicação entre diferentes aplicações e serviços

A comunicação entre diferentes aplicações e serviços apoiada por APIs estruturadas ocorre por meio de padrões de interações que podem ser de natureza síncrona ou assíncrona. Na comunicação síncrona, amplamente construída por meio de arquiteturas como REST, SOAP ou GraphQL, o cliente envia uma requisição estruturada e deve aguardar a resposta em tempo real do servidor, permanecendo bloqueado até que o processamento seja concluído e devolvido. Na comunicação assíncrona, ideal para cenários em que o cliente não demanda um retorno imediato da transação, o tráfego de dados e instruções ocorre por meio de mensageria intermediada

por corretores de mensagens (*message brokers*), nos quais comandos e eventos trafegam sem gerar bloqueios de execução nas aplicações clientes. Os comandos descrevem ações que se deseja realizar sobre uma entidade, ao passo que os eventos expressam reações lógicas disparadas em resposta a essas ações.

Em arquiteturas distribuídas e de microsserviços, a proliferação de múltiplas APIs de serviços especializados pode acarretar sérios problemas operacionais se os clientes tentarem consumi-las individualmente. Isso ocorre devido a fatores como o aumento da latência de rede decorrente de múltiplas verificações de firewall para acessar dados que estão divididos entre diferentes servidores de serviços e a necessidade de realizar chamadas consecutivas para montar uma única tela da interface de usuário. **Para mitigar esses problemas, adota-se o padrão de API Gateway, que funciona como o ponto de entrada único para o mundo externo, sendo responsável por efetuar o roteamento inteligente das requisições para os serviços apropriados, gerenciar a autenticação e realizar a chamada composição de APIs.** Esse processo de composição de APIs consiste em disparar requisições para diferentes APIs internas e unificar os resultados em um único objeto de transferência de dados (*Data Transfer Object* - DTO) otimizado e restrito ao escopo do que será exibido em tela para o cliente final.

2.4.4 Benefícios das integrações

Os benefícios decorrentes da integração profissional de sistemas e do estabelecimento de uma camada lógica de integração são expressivos para o desempenho operacional e governança tecnológica das organizações. Primeiramente, a integração elimina a necessidade de redigitação manual de dados entre sistemas distintos, o que reduz custos operacionais, atenua o retrabalho de funcionários e favorece a automação direta das atividades de negócios. Em segundo lugar, o tráfego íntegro de dados viabiliza a identificação e o monitoramento contínuo de eventos de negócios em tempo real, diminuindo o volume de exceções que demandam tratamento manual por operadores humanos. Além disso, a flexibilidade proporcionada pelas camadas integradoras baseadas em APIs permite que as empresas adaptem seus processos de forma ágil, segura e econômica, viabilizando novos modelos de negócios focados em relacionamento e facilitando o atendimento rápido a regulamentações de mercado e diretrizes lógicas.

Adicionalmente, a integração sistemática baseada em componentes permite a expansão consistente e segura das organizações, facilitando a consolidação de grandes portfólios de sistemas decorrentes de fusões e aquisições de outras empresas. Sob a perspectiva de infraestrutura, a divisão clara entre responsabilidades transacionais e o

gerenciamento lógico de processos de negócios viabiliza que as capacidades computacionais de cada aplicação evoluam de forma independente. **O uso de interfaces de comunicação desacopladas também é fundamental para maximizar o retorno dos grandes investimentos financeiros realizados na aquisição de softwares corporativos robustos (como sistemas ERP), que executam suas transações de forma altamente eficaz, mas operam de maneira isolada e sem um encadeamento lógico natural.** Por fim, em setores cruciais como o de saúde pública, a unificação de bases de dados fragmentadas e a orquestração de sistemas de informação via barramentos integrados trazem benefícios fundamentais ao cidadão, como maior segurança assistencial, qualificação das decisões clínicas em tempo oportuno, melhoria na tomada de decisões gerenciais e garantia da integralidade do cuidado oferecido.

2.5 FERRAMENTAS PARA APIs

O ciclo de vida e a governança de engenharia de software voltada para APIs exigem a adoção de ferramentas especializadas para o planejamento, design, desenvolvimento, testes e documentação das interfaces de comunicação. A existência de uma API se justifica quando há consumidores que a utilizam, o que impõe a necessidade técnica de fornecer informações exatas e padronizadas sobre seu funcionamento. No desenvolvimento moderno de sistemas distribuídos e microsserviços, a validação de conformidade com contratos, a avaliação de desempenho, a simulação de respostas e a automatização de testes representam áreas de relevância crítica. As ferramentas de software exercem a finalidade de mitigar falhas em tempo de execução, otimizar o fluxo de rede e garantir a consistência das entradas, saídas, tratamentos de erros e envelopes de resposta entre diferentes equipes de desenvolvimento.

2.5.1 Postman

O Postman consiste em um cliente de software amplamente utilizado na indústria tecnológica para criar, testar, gerenciar e realizar requisições lógicas a APIs baseadas no protocolo HTTP. Ele fornece uma interface interativa que permite aos desenvolvedores estruturar payloads, configurar cabeçalhos, definir tokens de autenticação e disparar requisições em variados verbos semânticos HTTP, analisando

os retornos e as mídias resultantes. No desenvolvimento e teste de APIs síncronas, o Postman permite a validação empírica de múltiplos padrões e formatos de dados. Exemplos práticos documentados nas fontes demonstram a sua utilização para efetuar chamadas transacionais de conversão de temperatura em APIs estruturadas sob o protocolo SOAP com arquivos XML, bem como para executar consultas dinâmicas de buscas (*queries*) com alteração de campos em APIs GraphQL baseadas em JSON.

No contexto de testes de software automatizados e de engenharia de confiabilidade, o Postman assume uma função estratégica na garantia da conformidade contratual entre serviços distribuídos. Para validar se a comunicação entre cliente e servidor ocorre estritamente de acordo com as especificações pactuadas, realiza-se o envio de requisições e a verificação do formato e conteúdo das respostas em busca de discrepâncias com o contrato estabelecido. O Postman suporta a automatização dessas rotinas por meio de coleções estruturadas denominadas **Postman Collections**, que permitem agrupar e sequenciar cenários de testes de conformidade contratual de forma automatizada, auxiliando no desenvolvimento ágil.

2.5.2 Swagger e OpenAPI

A especificação OpenAPI (OAS) e o ecossistema de ferramentas Swagger atuam de forma coordenada para descrever, documentar e expor as capacidades de APIs HTTP sem a necessidade de acesso direto ao seu código-fonte. A OpenAPI consiste em uma descrição de interface padrão independente de linguagem de programação, com suporte a formatos estruturados em JSON e YAML. Ao definir contratos de interface legíveis de forma clara e amigável para máquinas, ela se adapta a diversos processos de desenvolvimento (como o *Design-First* ou *API-First*), permitindo que humanos e computadores compreendam as capacidades de um serviço de maneira estável e padronizada.

O projeto Swagger reúne ferramentas de código aberto (*open-source*) projetadas para atuar em consonância com a especificação OpenAPI, dividindo-se em três soluções fundamentais de engenharia:

- **Swagger Editor:** Uma ferramenta baseada em navegador web que permite ao desenvolvedor modificar e visualizar a especificação da API em formato YAML ou JSON em tempo real.
- **Swagger UI:** Uma solução que interpreta a especificação OpenAPI e gera automaticamente uma interface web visual e totalmente interativa, por meio da qual desenvolvedores e usuários finais conseguem visualizar as capacidades

lógicas dos *endpoints*, os parâmetros requeridos, os formatos de payload esperados e realizar testes de chamadas síncronas diretamente pela tela.

- **Swagger Codegen:** Uma ferramenta cuja finalidade é processar a definição da API fornecida pelo contrato OpenAPI e gerar automaticamente servidores *stub* (servidores de simulação que emulam localmente os comportamentos e respostas do servidor real) e kits de desenvolvimento de software (SDKs) de clientes em dezenas de linguagens de programação.

Em ambientes corporativos baseados na plataforma de desenvolvimento .NET da Microsoft, a integração e a geração automatizada dessa especificação OpenAPI são impulsionadas pela biblioteca *open-source* **Swashbuckle**. Essa biblioteca é inserida diretamente no pipeline de execução da aplicação para mapear as classes controladoras (*controllers*) e os endpoints de rota da API, atualizando e gerando de maneira dinâmica os arquivos de documentação OpenAPI do Swagger de forma transparente para a equipe de engenharia.

2.5.3 Ferramentas de desenvolvimento, testes e documentação

A engenharia moderna de APIs e microsserviços apoia-se em um conjunto rico de ferramentas, bibliotecas e ambientes de execução para garantir a qualidade de software, a automação de processos e o provisionamento estável de serviços. No que se refere à modelagem e design estruturado de APIs REST, destaca-se a linguagem de modelagem baseada em YAML conhecida como **RAML (RESTful API Modeling Language)** e seu conjunto de soluções digitais. O ecossistema RAML engloba a ferramenta web *API Designer*, voltada para o design e compartilhamento de definições de APIs; o ambiente de desenvolvimento leve *API Workbench* (uma IDE depreciada para o gerenciamento de todo o ciclo de vida); e a solução *API Notebook*, que funciona como um ambiente web para a aceleração de designs e realização de testes interativos.

Para a validação de conformidade de contratos e detecção de discrepâncias físicas com relação às especificações acordadas, as fontes científicas de desenvolvimento ágil citam o uso de ferramentas como o **Pactflow** e a biblioteca de validação **Joi**, as quais automatizam testes a partir de arquivos contratuais, bem como à integração sistemática fornecida pela ferramenta **Spring Cloud**. No cenário do protocolo SOAP, destaca-se o uso do cliente de testes **SoapUI** para modelar e validar o envio de requisições estruturadas baseadas em envelopes XML e mapear *namespaces*.

No contexto do ecossistema .NET de desenvolvimento de APIs, bibliotecas especializadas estendem os recursos do framework para diferentes abordagens de intercomunicação síncrona:

- **HotChocolate:** Biblioteca amplamente empregada para configurar servidores GraphQL com tipagem forte de maneira simplificada, automatizando a injeção de dependências do contexto do banco de dados e a resolução das consultas.
- **Banana Cake Pop:** Interface interativa web disponibilizada em conjunto com o HotChocolate na rota padrão `/graphql` do servidor, utilizada para consultar a documentação de schemas autodescritivos e testar requisições de *queries* de forma dinâmica.
- **Grpc.AspNetCore:** Pacote .NET que disponibiliza todas as ferramentas e geradores de código necessários para a criação de serviços baseados no gRPC e compilação de arquivos *protocol buffer* (*protobuf*), os quais utilizam o compilador oficial *protoc* para serializar dados leves em formato binário de máquina.
- **Ocelot:** Biblioteca de código aberto utilizada para implementar soluções de *API Gateway* no pipeline do .NET, configurando roteamentos inteligentes de caminhos e rotas lógicas (*downstreams* e *upstreams*), autenticações e composições de APIs a partir de arquivos globais de configuração em formato JSON.

Por fim, os processos de entrega contínua (DevOps) e a integridade do ambiente transacional são assegurados pelo uso de tecnologias de virtualização e de automação de controle de fluxo de trabalho. A ferramenta de virtualização por contêineres **Docker** é extensamente aplicada para criar ambientes isolados que executam de forma ágil tanto as APIs quanto seus bancos de dados relacionais (como o PostgreSQL), reduzindo inconsistências ambientais. O gerenciamento dessas atividades de entrega, o controle de cartões de tarefas (via painéis Kanban com status *To Do*, *Doing* e *Done*), o versionamento e armazenamento de códigos-fonte e a publicação das versões no servidor de produção (como na nuvem Azure) são unificados na plataforma proprietária **Azure DevOps**. Complementarmente, o gerenciamento de estruturas de banco de dados e a geração automatizada de scripts DDL/SQL de migração são auxiliados por ferramentas de comparação integradas à IDE **Visual Studio Community**.

2.6 EXEMPLOS DE UTILIZAÇÃO DE APIs

O emprego de Interfaces de Programação de Aplicação (APIs) na engenharia de software é exemplificado em múltiplos segmentos tecnológicos e de negócios, demonstrando como essas interfaces viabilizam a troca de informações entre sistemas heterogêneos de forma segura e padronizada. Em vez de as equipes de desenvolvimento construírem aplicações isoladas ou estabelecerem acoplamentos físicos rígidos, as APIs fornecem as pontes contratuais e técnicas necessárias para conectar canais de comunicação estáveis em tempo de execução. As subseções a seguir apresentam exemplos práticos e reais documentados pelas fontes, descrevendo como as APIs são estruturadas e integradas para dar suporte a operações web, móveis, financeiras, geográficas, sociais e corporativas complexas.

2.6.1 Aplicações web

Aplicações web utilizam APIs extensamente para desvincular a lógica de exibição de dados no cliente (*frontend*) da lógica de processamento e persistência de dados (*backend*) hospedada em servidores remotos. Um exemplo real e detalhado presente na literatura de engenharia de software é o sistema “Apoio à Revisão Sistemática” (sysrev.azurewebsites.net), desenvolvido para otimizar a identificação e remoção de registros acadêmicos duplicados oriundos de bases de dados científicas de saúde. A arquitetura dessa aplicação web adota o modelo de aplicação de página única (*Single Page Application* - SPA), na qual o *frontend* foi implementado utilizando o framework Angular em TypeScript. A interface de usuário (UI) realiza requisições dinâmicas assíncronas via protocolo seguro HTTPS para uma API estruturada sob o padrão arquitetural REST, codificada em linguagem C# com o framework ASP.NET Core Web API. Essa API atua processando as solicitações e intermediando o acesso ao banco de dados relacional Microsoft SQL Server. Por meio de chamadas síncronas que trafegam representações de dados estruturadas no formato leve JSON, o cliente web consome e envia dados em tempo de execução para realizar operações lógicas como importação de resumos literários, filtragem de dados e acionamento de algoritmos pesados de remoção de duplicatas no servidor, sem que o código do *backend* precise ser exposto ou executado na máquina local do usuário. Outro modelo de aplicação web apontado pelas fontes é a simulação didática de catalogação de livros, em que o cliente web consome dados síncronos realizando requisições GET para endpoints específicos (como <http://www.biblioteca.com/api/v1/autores/32/livros>) para receber e renderizar listagens em tela no formato JSON.

2.6.2 Aplicativos móveis

No desenvolvimento de aplicativos móveis, o uso de APIs é crucial em razão da necessidade de otimização de consumo de bateria, largura de banda e limitação física de processamento dos dispositivos celulares, que tipicamente demandam menores fluxos de dados do que ambientes web clássicos. Um exemplo de destaque no setor público brasileiro é o aplicativo "Conecta Recife", mantido pela Prefeitura de Recife para colocar na palma da mão do cidadão o acesso unificado a centenas de serviços municipais. Esse aplicativo móvel consome APIs integradas diretamente à plataforma municipal de interoperabilidade de saúde. Por meio do consumo de APIs web seguras, o aplicativo disponibiliza dados pessoais e clínicos do usuário, como o histórico de vacinação e o prontuário unificado do cidadão ("Minha Saúde Conectada") de forma rápida e segura. Para suprir de forma eficiente as conexões instáveis de rede móvel (3G/4G/5G), as fontes técnicas de microsserviços recomendam a utilização de APIs integradas ao padrão arquitetural de API Gateway. Nesse modelo de software, um gateway dedicado para dispositivos móveis (como o "Mobile Compras") intercepta uma chamada única enviada pelo aplicativo móvel, orquestra as comunicações internas necessárias entre múltiplas APIs de microsserviços do backend (como produto, catálogo e identidade), unifica os diferentes dados em um objeto de transferência (*Data Transfer Object* - DTO) otimizado e devolve um único retorno ao celular, minimizando o tráfego de rede e a latência de comunicação.

2.6.3 Sistemas de pagamento

As fontes demonstram a utilização de APIs para integrar fluxos comerciais de vendas a plataformas de processamento financeiro e de pagamentos de maneira transparente e desacoplada. Em uma modelagem clássica de microsserviços aplicada a uma loja virtual, por exemplo, o fluxo de fechamento de um pedido por parte de um cliente consome as capacidades do sistema de forma independente. Após o preenchimento dos dados lógicos da compra no carrinho de compras e o acionamento do comando de "anotar pedido", a API de Pedidos do backend dispara um evento transacional. Esse evento trafega através de um corretor de mensagens (*message broker*), como o RabbitMQ, contendo informações lógicas sobre a venda para que sejam recebidas pela API de Pagamentos. A API de Pagamentos, atuando de maneira assíncrona, intercepta a notificação, executa o processamento do comando de validação do pagamento (*validarPagamento*) e devolve um evento de confirmação para o barramento de integração. Esse uso de APIs e chamadas assíncronas via mensageria

garante que o processamento do pagamento ocorra em segundo plano sem travar a navegação do usuário e sem gerar dependência física, evitando que falhas temporárias na API de cobrança impactem a disponibilidade e o desempenho da API de vendas de produtos principais.

2.6.4 Mapas e localização

A utilização de APIs voltadas para controle de dados geográficos e recursos de localização é documentada na literatura técnica por meio do gerenciamento de infraestrutura física de rede de serviços. No caso corporativo da empresa de telecomunicações GVT, o sistema de informação geográfica (GIS), denominado SAGRE e desenvolvido pelo CPqD, foi integrado à arquitetura computacional da empresa para o mapeamento da rede externa de telecomunicações. Durante o processo de atendimento de instalação de uma nova linha telefônica, quando um potencial cliente entrava em contato com o *call center*, o sistema de CRM (Siebel) executava uma chamada de procedimento de forma síncrona para a API do GIS para realizar a verificação física de cobertura do endereço cadastrado ("verifica cobertura"). Caso houvesse viabilidade técnica de localização geográfica, a API do GIS era novamente acionada pela camada de integração por meio do disparo do comando "aloca facilidade" para reservar o par físico de rede correspondente ao novo assinante. Posteriormente, após a finalização física dos trabalhos de campo e o encerramento do BA (Boletim de Atividade), a API do GIS recebia o evento de fechamento de ordem para alterar o status do par físico no mapa para "facilidade ocupada", mantendo o inventário geográfico em tempo real.

2.6.5 Redes sociais

Nas plataformas de redes sociais, o desenvolvimento e a concepção de novos padrões de APIs foram motivados pela necessidade extrema de escalabilidade técnica, dinamismo e grande flexibilidade no consumo de recursos. O principal exemplo real documentado reside na criação da tecnologia de APIs GraphQL pelo Facebook (atual Meta) no ano de 2012 (disponibilizada como código aberto em 2015). O Facebook deparava-se com as limitações de tráfego impostas pelas APIs REST convencionais de endpoints engessados, que causavam problemas severos de tráfego de dados para seus aplicativos celulares. Com a API GraphQL, desenhada de forma centrada no produto a partir da visão das equipes de *frontend*, a plataforma passou a expor seus schemas autodescritivos e sua estrutura em grafos. Essa API permite que a rede social receba as

consultas e retorne em tempo de execução apenas as chaves e valores especificamente requisitados pelo cliente, eliminando desperdícios (*over-fetching* e *under-fetching*) e superando gargalos de infraestrutura de conectividade móvel.

2.6.6 Sistemas corporativos

Os exemplos reais mais complexos e abrangentes de utilização de APIs na integração de sistemas corporativos ocorrem por meio de barramentos de serviços de grande porte, tanto na saúde pública quanto em telecomunicações. No setor de saúde, destaca-se a federação de dados clínicos implementada entre o município de Recife (Plataforma Saúde Conectada) e a Ebserh, que unificou dados de mais de 25 milhões de usuários e 45 hospitais universitários federais. Nesse ecossistema de saúde digital, são consumidas APIs web baseadas em microserviços sob os padrões internacionais de interoperabilidade HL7 FHIRv4 e OpenEHR. Conectores de software e APIs específicas são utilizados para orquestrar e cruzar as informações clínicas de 12 sistemas corporativos distintos. Através das APIs, são intercambiados de maneira ágil prontuários eletrônicos do cidadão (e-SUS APS e AGHU), regulação assistencial (via API do Sisreg), imunizações (API do Sipni), resultados de exames de laboratório (LISNet), assistência farmacêutica (Horus-Recife) e notificações de agravos (Sinan).

Outro caso corporativo relevante de utilização de APIs é o projeto de Business Process Integration (BPI) da GVT, no qual uma camada integradora de gerenciamento de mensagens (Vitria) interconectou sistemas corporativos críticos e heterogêneos de diferentes fornecedores de pacotes de mercado. O ecossistema integrou o CRM da Siebel, o faturamento Arbor (da CSG Systems), o provisionamento de rede interna Metasolv, o GIS SAGRE (CPqD), o gerenciamento de equipes SGE (CPqD) e as próprias centrais telefônicas físicas (*SWITCH*). A comunicação dessas APIs e microserviços via middleware permitiu que a transação para ativar uma nova linha caísse de dias para minutos, com corte de 60% nos esforços de trabalho manuais de provisionamento e diminuição de 80% das reclamações de usuários finais por inconsistências lógicas no banco de dados.

2.7 VANTAGENS E DESAFIOS DAS APIs

A incorporação de Interfaces de Programação de Aplicação (APIs) no desenvolvimento e na engenharia de software contemporânea oferece uma ampla gama de oportunidades técnicas, ao mesmo tempo em que impõe restrições lógicas e novos desafios de governança para as equipes de tecnologia. Como canais formais de comunicação síncrona ou assíncrona, as APIs permitem que sistemas de informação colaborem eletronicamente. Contudo, a transição de arquiteturas monolíticas para modelos distribuídos baseados em microsserviços e APIs exige uma análise criteriosa sobre os prós e contras dessas abordagens em termos de viabilidade financeira, esforço de programação e riscos de falhas sistêmicas na rede.

2.7.1 Reutilização e interoperabilidade

A principal vantagem decorrente da adoção de APIs reside na promoção da interoperabilidade de dados e no fomento à reutilização de componentes lógicos de software. A interoperabilidade é definida como a habilidade de dois ou mais sistemas ou componentes trocarem informações e serem capazes de utilizá-las de maneira mútua e útil. No contexto acadêmico, esse processo distingue-se da integração tradicional ponto a ponto, uma vez que a integração física gera um acoplamento estreito e dependência tecnológica recíproca entre os sistemas envolvidos, ao passo que a verdadeira interoperabilidade apoia-se em APIs padronizadas para viabilizar a troca de dados sem estabelecer vínculos tecnológicos rígidos. A utilização de APIs atua como uma barreira contra o fenômeno da "integração do tipo espaguete", em que múltiplas conexões locais de rede são entrelaçadas confusamente, o que tradicionalmente resultava na "perpetuação do legado". Ao expor recursos de negócios como componentes reutilizáveis baseados em especificações claras (como RAML ou OpenAPI), as organizações conseguem reaproveitar lógicas e códigos em diferentes projetos, mantendo a consistência estrutural das entradas, saídas e tratamentos de erros de software.

2.7.2 Escalabilidade

As características de projeto das APIs fornecem suporte direto para que a capacidade operacional e de processamento dos sistemas de informação seja estendida continuamente. Sob a perspectiva arquitetural, Fielding estabelece que a separação de responsabilidades na relação cliente-servidor eleva a portabilidade das interfaces de usuário e viabiliza a escalabilidade do servidor de forma independente por meio da redução de sua complexidade interna. Essa escalabilidade é potencializada

pelo cumprimento da restrição de comunicação sem estado (stateless), a qual determina que cada requisição contenha individualmente todos os dados de contexto necessários para seu processamento no servidor; isso simplifica drasticamente a alocação e a liberação de recursos de hardware nos servidores, uma vez que estes não precisam gerenciar dados de sessão ativos dos clientes. Na engenharia de software voltada para a web, o uso do conceito de aplicações de página única (Single Page Applications - SPAs) que interagem com APIs REST ajuda a manter a escalabilidade geral ao transferir o processamento das regras de exibição e entrada de dados diretamente para o navegador do dispositivo do usuário final (computador ou celular), reduzindo a carga computacional redundante no servidor. Ademais, ambientes de produção modernos baseados em computação em nuvem estruturam-se para aumentar ou diminuir essa infraestrutura de servidores de APIs e bancos de dados automaticamente conforme as oscilações de demanda de acesso em tempo real, cobrando custos operacionais proporcionais aos recursos de hardware efetivamente consumidos.

2.7.3 Segurança

A preservação da confidencialidade, integridade e privacidade dos dados representa um fator crítico de sucesso (FCS) e um dos desafios mais complexos no ciclo de vida de projetos que expõem APIs. Conforme as exigências legais contemporâneas, como a Lei Geral de Proteção de Dados (LGPD) no Brasil, as organizações de TI são obrigadas a estruturar o controle de acesso e autenticação de forma rigorosa para resguardar as informações pessoais dos cidadãos contra vazamentos e incidentes de segurança. Para assegurar o tráfego íntegro na rede de computadores, as transações devem ocorrer obrigatoriamente de maneira criptografada utilizando-se o protocolo seguro HTTPS combinado a certificados Secure Socket Layer (SSL), que codificam as informações e impedem que terceiros não autorizados as capturem na rede. Adicionalmente, as APIs utilizam o padrão de autorização baseado em tokens criptográficos, como o Json Web Token (JWT), no qual as permissões lógicas do usuário são encapsuladas temporariamente em um token programado para expirar após determinado período, mitigando acessos indesejados. Esses cuidados são vitais para mitigar variados vetores de ameaças cibernéticas direcionadas às APIs públicas, as quais incluem tentativas diretas de violar barreiras de autenticação, a injeção de scripts e conteúdos maliciosos destinados a comprometer a base de dados, e ataques do tipo API Fuzzing, que consistem no envio de múltiplos parâmetros aleatórios e inconsistentes à interface para diagnosticar falhas e levantar informações internas sobre as vulnerabilidades físicas do sistema.

2.7.4 Manutenção e versionamento

O planejamento evolutivo e a manutenção de APIs demandam disciplina metodológica acentuada em razão do risco de que alterações lógicas nos endpoints interrompam abruptamente as operações de sistemas clientes em tempo de execução. Como as APIs são projetadas uma vez, mas chamadas de forma massiva pelos clientes em rede, mesmo falhas de design consideradas menores ou inocentes tendem a ser ampliadas desproporcionalmente, manifestando-se em cada ponto de chamada e gerando severos danos colaterais ao sistema. Para contornar esses gargalos de integração, a adoção de abordagens como o Design-First (ou API-First) destaca-se como boa prática, pois garante que o contrato de interface seja detalhado, validado e acordado antes de sua codificação, reduzindo sensivelmente os retrabalhos futuros. Quando modificações lógicas no contrato tornam-se indispensáveis ao longo do ciclo de vida em ambiente de produção, o controle de versão surge como recurso estratégico para guiar a depreciação ordenada e segura de recursos antigos. Nesse cenário, enquanto as APIs REST apoiam-se em modelos de versionamento numérico explícito (tipicamente exposto na própria URL de acesso, como no caminho `"/v1/"`), o que impõe aos clientes o desafio manual de descobrir se estão utilizando a versão mais recente, a arquitetura GraphQL flexibiliza esse controle por meio de esquemas (schemas) autodescritivos contínuos que funcionam de forma análoga ao controle de versões do repositório Git, viabilizando evoluções integradas sem a necessidade de fragmentação rígida por números.

2.7.5 Dependência de serviços externos

A utilização de APIs públicas ou de soluções disponibilizadas por terceiros acentua a agilidade no desenvolvimento, mas gera vulnerabilidades associadas à dependência de serviços externos, especialmente quando não há o controle direto das modificações aplicadas à interface. Em cenários corporativos que demandam a integração com sistemas legados externos ou fornecedores de software monolíticos específicos, a dependência excessiva pode ocasionar a perpetuação dessas tecnologias obsoletas, elevando os custos de governança e inviabilizando financeiramente o ecossistema colaborativo. Do ponto de vista técnico, em comunicações síncronas que exigem respostas imediatas, a latência de tráfego, as lentidões de rede celular em canais móveis ou as falhas temporárias no servidor da API externa comprometem diretamente a estabilidade das aplicações clientes. Para mitigar o risco de indisponibilidade sistêmica e evitar que problemas em uma API externa causem falhas em cascata que derrubem a aplicação inteira, a engenharia de software

recomenda a implementação de padrões de resiliência e tratamento de erros nas conexões. Entre essas técnicas, destaca-se o padrão Retry, o qual gerencia a execução de novas tentativas automáticas de requisição síncrona após a detecção de uma falha inicial de conexão, e o padrão Circuit Breaker (disjuntor), que interrompe temporariamente o fluxo de chamadas para o serviço externo após a ocorrência de determinado número de falhas consecutivas, mantendo o sistema cliente em quarentena de requisições por um intervalo pré-definido até que o serviço de terceiros se restabeleça.

3 CONCLUSÃO

A análise sistemática desenvolvida ao longo deste trabalho permite concluir que as Interfaces de Programação de Aplicação (APIs) consolidaram-se como elementos de infraestrutura tecnológica indispensáveis no cenário contemporâneo de engenharia de *software*. Conceitualmente caracterizadas como conjuntos de regras e protocolos formais de comunicação, as APIs funcionam essencialmente como contratos lógicos que determinam de forma padronizada os termos de intercâmbio de dados e recursos entre dois serviços heterogêneos. A finalidade precípua dessas interfaces consiste em prover uma camada robusta de abstração e segurança, ocultando a complexidade lógica e física interna de sistemas e microserviços para expor somente os *endpoints* necessários para o seu consumo.

Estruturalmente, o funcionamento básico das APIs baseia-se no fluxo contínuo de envio de requisições por um cliente e retorno de respostas por um servidor. Essa intercomunicação síncrona ou assíncrona baseia-se no mapeamento de *endpoints* associados a métodos de controle semântico HTTP (como GET, POST, PUT e DELETE) e utiliza linguagens de formatação de dados textuais e legíveis como JSON e XML. Entre as arquiteturas estudadas, o padrão REST destaca-se por sua flexibilidade fundamentada em restrições como comunicação sem estado (*stateless*) e interface uniforme, embora apresente limitações de *payloads* inflexíveis. Em contrapartida, o protocolo SOAP impõe uma rigidez maior por meio do encapsulamento em envelopes XML pesados de acesso único, enquanto o GraphQL surge como uma alternativa ágil que resolve problemas

clássicos de desperdício de dados (*over-fetching* e *under-fetching*) ao permitir que o cliente determine dinamicamente por meio de consultas em grafos quais campos devem ser retornados em uma única chamada.

No processo de integração de sistemas, as APIs exercem o papel crítico de "chamadas" (*call interfaces*), atuando de forma estratégica para unificar e orquestrar fluxos transacionais heterogêneos de maneira desacoplada, o que viabiliza a interoperabilidade sem a imposição de dependências tecnológicas rígidas. Ao substituir as conexões confusas características da abordagem de "integração espaguete", as APIs defendem a arquitetura corporativa contra a indesejada "perpetuação do legado". A literatura empírica examinou ferramentas fundamentais que acompanham esse ciclo, como o Postman para validação e testes automatizados de contratos, e a especificação OpenAPI pelo Swagger e RAML para *design-first* e geração automatizada de documentação interativa e SDKs. Casos práticos demonstraram que esses conceitos produzem resultados expressivos, a exemplo do sistema web sysrev (hospedado na nuvem Azure com *frontend* Angular e *backend* REST em .NET Core); o barramento de saúde digital SUS de Recife e Ebserh, que unificou dezenas de prontuários eletrônicos sob padrões HL7 FHIR e OpenEHR via federação de plataformas; e o projeto de BPI da operadora GVT, que reduziu o tempo de ativação de novas linhas de dias para minutos por meio de *middleware* e conectores de APIs.

Finalmente, a adoção de APIs carrega consigo um conjunto nítido de vantagens e desafios que devem pautar a tomada de decisão em projetos de ADS. Como vantagens operacionais proeminentes, ressaltam-se o estímulo à reusabilidade de regras de negócios, a escalabilidade distribuída impulsionada por padrões *stateless*, e a segurança proporcionada por criptografia HTTPS/SSL e autorização forte via tokens JWT. Já os principais desafios abrangem as complexidades de *design* e controle de versão de *endpoints* para evitar rupturas, os riscos cibernéticos que exigem monitoramento contra ataques e testes rigorosos, o *overhead* computacional de processamento (como o motor GraphQL que, embora apresente desempenho 2701% superior ao REST em cenários de múltiplas entidades relacionadas, acarreta uma perda de 11% a 14% em requisições consolidadas simples de *endpoint* único), e a vulnerabilidade decorrente da dependência de serviços externos, contornável apenas por meio de padrões resilientes de *software* como *Circuit Breakers* e *Retry*. Conclui-se, portanto, que a formulação de uma estratégia profissional de APIs constitui um pilar essencial para o desenvolvimento de *software* ágil e para a consolidação de ecossistemas corporativos integrados, conferindo agilidade e longevidade aos sistemas de informação nas organizações modernos.

4 REFERÊNCIAS

ANASTÁCIO, Wellington Montefusco; PRADO, Edmir Parada Vasques. Fatores críticos de sucesso na integração de sistemas: um estudo de caso em uma organização pública brasileira. In: SIMPÓSIO BRASILEIRO DE SISTEMAS DE INFORMAÇÃO – SBSI, 2014. Anais [...]. Londrina: UEL, 2014. Disponível em: <https://sol.sbc.org.br/index.php/sbsi/article/view/6103/6001>. Acesso em: 14 ago. 2026.

CRUZ, Julio da Silva. Modelagem de microsserviços e arquiteturas de APIs. 2021. Trabalho de Conclusão de Curso (Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas) – Faculdade de Tecnologia de São Paulo, São Paulo, 2021. Disponível em: https://ric.cps.sp.gov.br/bitstream/123456789/8677/1/ads_2021_2_juliodasilvacruz_modelagemdemicroservi%C3%A7osearquiteturadeapis.pdf. Acesso em: 14 ago. 2026.

DE SORDI, José Osvaldo; MARINHO, Bernadete de Lourdes. Análise dos ambientes para integração entre sistemas de informação segundo especialistas. Revista de Ciências da Administração, Florianópolis, 2005. Disponível em: <https://periodicos.ufsc.br/index.php/adm/article/view/1678/1423>. Acesso em: 14 ago. 2026.

DE SORDI, José Osvaldo; MEDEIROS JÚNIOR, Gildo. Abordagem sistêmica para integração entre sistemas de informação e sua importância à gestão da operação: análise do caso GVT. Gestão & Produção, São Carlos, v. 13, n. 1, p. 105-116, 2006. DOI: 10.1590/S0104-530X2006000100010. Disponível em: <https://www.scielo.br/j/gp/a/Tz6ccWKtt4hqkWp9MwGTW9q/?format=html&lang=pt>. Acesso em: 14 ago. 2026.

NASCIMENTO SILVA, Patrícia; DA SILVA, Gabriel Vieira Pereira. Recuperação de dados abertos em APIs: consumindo dados agregados do IBGE. Encontros Bibli: revista eletrônica de biblioteconomia e ciência da informação, Florianópolis, v. 29, p. 1-17, 2023. DOI: 10.5007/1518-2924.2024.e96185. Disponível em: <https://periodicos.ufsc.br/index.php/eb/article/view/96185>. Acesso em: 14 ago. 2026.

NETO, Valter Granato. Uma análise comparativa de desempenho entre APIs REST e GraphQL em cenários com múltiplas entidades. 2025. 55 p. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Universidade Federal de Lavras, Lavras, 2025. Disponível em: https://sip.prg.ufla.br/publico/trabalhos_conclusao_curso/acessar_tcc_por_curso/ciencia_da_computacao/index.php?dados=20242201920240. Acesso em: 14 ago. 2026.

PINTO, Hêider Aurélio; SANTANA, José Santos Souza; LIMA, Ana Maria Freire de Souza; CHIORO, Arthur. Superando a histórica fragmentação de dados no SUS: interoperabilidade em Recife e na Ebserh. Saúde em Debate, Rio de Janeiro, v. 49, n. especial 1, 2025. DOI: 10.1590/2358-28982025E110011P. Disponível em: <https://www.scielo.br/j/sdeb/a/6W8bBwjTQQhKzpvZnrPpdwm/?lang=pt>. Acesso em: 14 ago. 2026.

SISTEMA “Apoio à Revisão Sistemática”. Revista Brasileira de Epidemiologia, São Paulo, v. 25, e220030, 2022. Disponível em: <https://www.scielo.br/j/rbepid/a/CTDF8fHhCqbMcvzDcZJTLwb/?format=pdf&lang=pt>. Acesso em: 14 ago. 2026.